

牛顿法与拟牛顿法

[牛顿法.pdf](#)

从“梯度下降只看斜率”出发，引出“牛顿法同时看斜率和曲率”，用二阶 Taylor 模型构造 Newton 方向，再讨论它为什么快、为什么可能不稳定，以及如何用阻尼、修正和近似求解让它可用。

二阶近似于Newton方向：

首先回顾，无约束优化问题是： $\min_{x \in \mathbb{R}^n} f(x)$ 。梯度下降法的迭代格式为：

$$x^{k+1} = x^k - \alpha_k \nabla f(x^k).$$

其中， $\nabla f(x^k)$ 是当前位置的梯度，表示函数值上升最快的方向；所以 $-\nabla f(x^k)$ 就是函数局部下降最快的方向。梯度下降的核心思想就是：**每一步沿着负梯度方向走一小步。**

核心瓶颈

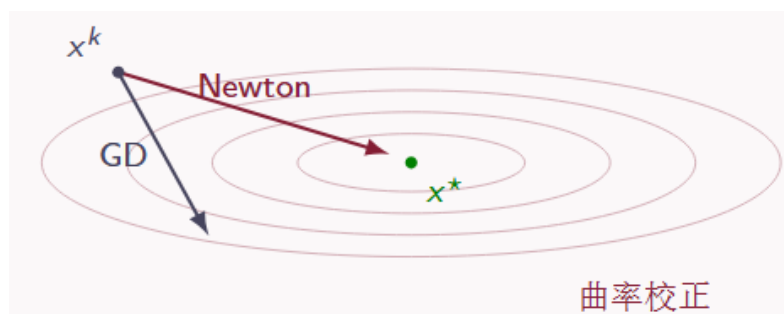
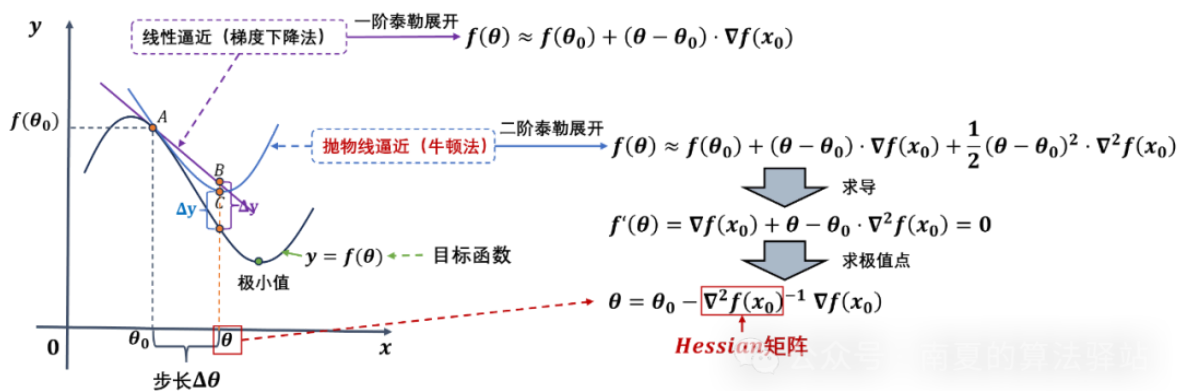
当目标函数的水平集非常狭长时，负梯度方向可能频繁横穿谷底，导致 zig-zag 和慢收敛。

就比如下面这个图，这个二次函数的**条件数很大的**，所以图形明显是一个非常陡峭的，如果用梯度下降，GD算法，可以看到黑色箭头会多次 zig-zag，**而牛顿法就可以根据曲率一次找到最优方向（当二次函数时）**

推论 3.1: 严格凸二次函数

对任意初始点 x^0 ，标准牛顿法在严格凸二次函数上一步达到全局最优解。

因为牛顿法不是简单地沿负梯度走，而是**先在当前点附近搭一个二次模型**，然后走向这个二次模型的最低点。**二次函数时，这个模型等于原函数本身**，所以一步到最优点；一般函数时，它只是局部近似，所以需要线搜索或阻尼来防止走太猛。



二阶 Taylor 近似为:

$$f(x^k + d) \approx f(x^k) + \nabla f(x^k)^T d + \frac{1}{2} d^T \nabla^2 f(x^k) d$$

d 是“从当前点出发要走的位移向量”; $\nabla f(x^k)$ 是梯度, 表示一阶斜率; $\nabla^2 f(x^k)$ 是 Hessian 矩阵, 表示不同方向上的弯曲程度。

为了简化记号, 通常记 $g_k = \nabla f(x^k)$, $G_k = \nabla^2 f(x^k)$, 于是构造局部二次模型:

$$m_k(d) = f(x^k) + g_k^T d + \frac{1}{2} d^T G_k d$$

站在当前点 x^k , 用一个二次函数预测“如果我走 d , 函数值会变成多少”。接下来, 我们就是要找到让**局部二次模型最小**的那个位移向量。

对 $m_k(d)$ 关于 d 求导, 得到 $\nabla_d m_k(d) = g_k + G_k d$ 。令导数为 0, 就得到 Newton 方程 $G_k d_k = -g_k$ 。如果 G_k 可逆, 则 $d_k = -G_k^{-1} g_k$ 。

优化视角: 在 x^k 附近用一个抛物线近似 f , 抛物线的最低点就是下一点 x^{k+1}

何时是下降? :

Newton 方向满足 $d_k = -G_k^{-1} g_k$ 。如果 $G_k \succ 0$, 也就是 **Hessian 正定**, 那么 G_k^{-1} 也正定, 于是 $g_k^T d_k = -g_k^T G_k^{-1} g_k < 0$, 只要 $g_k \neq 0$, **这个方向就是下降方向。**

Hessian 正定意味着局部二次模型像一个向上开口的碗。既然这个碗有明确的最低点, 那么从当前点走向这个最低点, 自然是下降的。

定义 2.2: Newton 减量

当 $G_k \succ 0$ 时, 定义

$$\lambda_k^2 = g_k^T G_k^{-1} g_k = -g_k^T d^k.$$

它刻画当前点在局部二次模型下距离最优的程度。

牛顿法的一些风险情况:

牛顿法依赖的是**局部二次模型**, 如果一开始就离极小点有点远, 那么模型可能给出不理想的预测:

对

$$f(x) = x^4 - 3x^2$$

有两个局部极小点 $x = \pm\sqrt{3/2}$, 中间 $x = 0$ 是局部极大点。

取 $x^k = 0.5$, 则

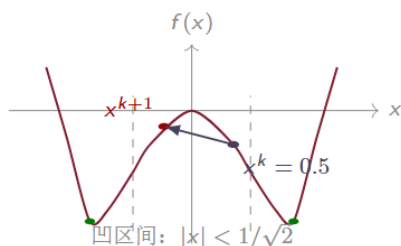
$$f'(0.5) = -2.5, \quad f''(0.5) = -3.$$

Newton 方向为

$$d^k = -\frac{f'(0.5)}{f''(0.5)} = -\frac{-2.5}{-3} \approx -0.833.$$

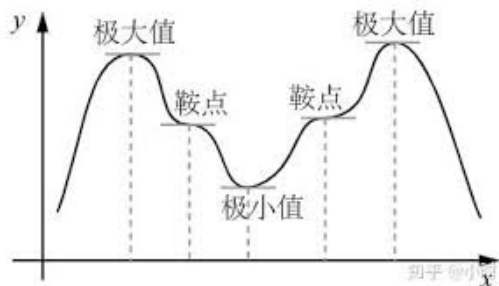
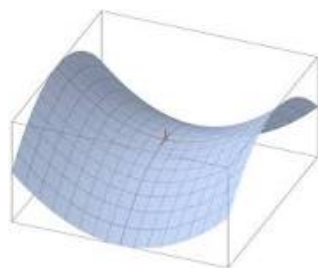
判断是否下降要看

$$f'(x^k)d^k \approx (-2.5)(-0.833) > 0.$$



很显然函数值不仅没有下降反而上升!!

对于一些非凸的函数, 梯度为 0 的点并不总是极小点, 只要是驻点就有梯度为 0, 那么这个点还有可能是**鞍点 or 极大点**;



同时, 我们也知道, 海森矩阵正定的时候方向是下降的, 但是海森矩阵**不正定的时候**, 牛顿方程依然是能解出来解的, 这种情况下, **方向不一定对应着下降方向。**

(因为如果海森矩阵不正定，那就代表着有特征值是负的，负的特征值导致的方向可能朝着上升的地方)

教学要点

Newton 法本质上是在求解 $\nabla f(x) = 0$ ，因此它会快速逼近驻点；但在非凸问题中，驻点不必是局部极小点。

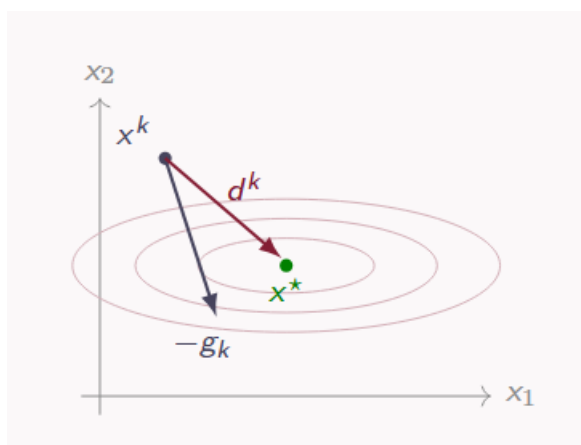
上面两个原因，都告诉我们牛顿法是存在一些**不稳定性和风险**的，他在用曲率优化解答的同时，也带来了“迷路”的可能。

牛顿方向几何解释：

Newton 方向可以看作经过曲率校正的一阶方向：

$$d^k = -G_k^{-1} g_k.$$

曲率大的地方步子会缩小，曲率小的方向步子会放大，对于二次函数，之恰好直接指向最优解，上面也从数学角度解释过了。



P14页有个例子很好，可以看一下。

标准牛顿法 and 阻尼牛顿法 and 修正牛顿法 and 信赖域

标准牛顿法：

标准牛顿法步长就是全步长，**不再在 d_k 前面乘 α 了**，这个方法最优点附近的时候很快，但是远离最优点的时候可能“跑过了”。

算法：标准牛顿法

给定初始点 x^0 。对 $k = 0, 1, 2, \dots$:

1. 计算 $g_k = \nabla f(x^k)$, $G_k = \nabla^2 f(x^k)$;
2. 解线性方程组 $G_k d^k = -g_k$;
3. 更新 $x^{k+1} = x^k + d^k$;
4. 若 $\|g_k\|$ 足够小, 则停止。

阻尼牛顿法:

可能问题

- ▶ 二次模型只在局部准确;
- ▶ Hessian 可能不是正定;
- ▶ Newton 方向可能不是下降方向。

常用修正

- ▶ 使用线搜索选择步长;
- ▶ 必要时修正 Hessian;
- ▶ 或使用信赖域方法。

阻尼牛顿法和标准牛顿法的区别就在于, **在方向前乘了一个 α 步长**, 这个 α 属于 $(1, 0]$ 。求方向 d_k 的方法不变, 依然由牛顿方程解出来。

Armijo 条件

选择 α_k 使

$$f(x^k + \alpha_k d^k) \leq f(x^k) + c \alpha_k \nabla f(x^k)^T d^k, \quad c \in (0, 1).$$

远离最优点时, 线搜索保证充分下降, 靠近最优点时, 通常会接受全步长, 即通常接受 $\alpha_k = 1$, 所以阻尼牛顿法具有**先稳健后快速**的特征。

这里选择步长的原则仍然是 *Armijo* 原则, 先从大步长开始, 不符合要求了回溯线搜索, 缩小步长, 直到满足 *Armijo* 条件为止。

修正牛顿法:

作用

- ▶ 提高矩阵正定性;
- ▶ 避免过大或不稳定步长;
- ▶ 使方向更接近下降方向。

直觉

μ_k 大时更像梯度下降; μ_k 小时更像标准 Newton。

其实非常简单粗暴, 如果海森矩阵不正定, 特征值有负的, 就加单位矩阵上去直到全为正。这样的方法可能导致曲率并不是“最优的”, 但至少保证了方向一定是下降的方向, **舍曲率保下降, 不完全相信原海森矩阵了。**

信赖域:

如果说线搜索是先确定方向, 我们在确定好的方向的基础上确定步长, 那信赖域就直接**限制了二次模型的适用范围。**

$$\min_d g_k^T d + \frac{1}{2} d^T G_k d, \quad \|d\| \leq \Delta_k.$$

这里 Δ_k 叫做**信赖域半径**。

通俗理解: 只相信当前点附近半径 Δ_k 以内的二次模型。我们可以在这个范围内找最优步子, 但不能跑到范围外。

- 模型预测与真实下降一致, 增大信赖与半径
- 模型预测不可靠, 缩小信赖域半径

策略	基本思想	适用特点
线搜索	先给出方向, 再选择步长 α_k	实现简单, 常与下降方向配合使用
信赖域	在可信半径内求解二次模型	对不定 Hessian 和非凸情形更稳健

共同目标

二者都是为了把局部二次模型转化为具有全局稳定性的迭代算法。

收敛性质与计算代价:

收敛性质:

牛顿法最重要的性质是局部二次收敛

定理 4.1: 局部二次收敛（非正式表述）

设 f 二阶连续可微, x^* 满足

$$\nabla f(x^*) = 0, \quad \nabla^2 f(x^*) \succ 0.$$

若 Hessian 在 x^* 附近 Lipschitz 连续, 并且初始点足够接近 x^* , 则标准牛顿法产生的序列满足

$$\|x^{k+1} - x^*\| \leq C \|x^k - x^*\|^2.$$

当误差已经比较小的时候, **下一步的误差大概与当前误差的平方同阶**, 下降速度超级超级快。

设 $e_k = \|x^k - x^*\|$ 。常见收敛速度可用误差序列描述:

类型	典型形式
线性收敛	$e_{k+1} \leq q e_k, \quad 0 < q < 1$
超线性收敛	$\frac{e_{k+1}}{e_k} \rightarrow 0$
二次收敛	$e_{k+1} \leq C e_k^2$

Newton 法的局部二次收敛意味着: 当 e_k 已经较小时, 误差会以近似“平方”的速度下降。

我们可以通过 Taylor 展开来证明: 对梯度函数 ∇f 做一阶 Taylor 展开。在 x^k 附近展开到 x^* , 得到:

$$0 = \nabla f(x^*) = \nabla f(x^k) + \nabla^2 f(x^k)(x^* - x^k) + r_k.$$

代入 $g_k = \nabla f(x^k)$ 、 $G_k = \nabla^2 f(x^k)$ 、 $x^* - x^k = -e_k$, 得到:

$$0 = g_k - G_k e_k + r_k$$

因此 $g_k = G_k e_k - r_k$, 说明当前梯度 g_k 的主要部分是 $G_k e_k$, 也就是“当前曲率矩阵 $\times$ 当前误差”; 剩下的 r_k 是 Taylor 余项。

这里 r_k 是 Taylor 余项。若 Hessian 在最优点附近 Lipschitz 连续, 则

$$\|r_k\| \leq C_1 \|x^* - x^k\|^2 = C_1 \|e_k\|^2.$$

Newton 更新给出

$$G_k d^k = -g_k, \quad x^{k+1} = x^k + d^k.$$

因此误差满足

$$e_{k+1} = x^{k+1} - x^* = e_k - G_k^{-1} g_k.$$

代入 $g_k = G_k e_k - r_k$:

$$e_{k+1} = e_k - G_k^{-1} (G_k e_k - r_k) = G_k^{-1} r_k.$$

若 x^k 足够接近 x^* , 则 G_k 可逆且 $\|G_k^{-1}\| \leq C_2$ 。于是

$$\|e_{k+1}\| \leq \|G_k^{-1}\| \|r_k\| \leq C \|e_k\|^2.$$

直观理解

Newton 步把主要的一阶误差 $G_k e_k$ 抵消掉, 下一步误差只来自 Taylor 余项, 所以是二阶小量。

因为 Newton 步本来是:

$$d^k = -G_k^{-1} g_k.$$

而因为 $g_k \approx G_k e_k$, 所以:

$$d^k \approx -G_k^{-1} G_k e_k = -e_k.$$

这说明 Newton 步大概就是:

$$d^k \approx -e_k.$$

而从 x^k 走 $-e_k$ 是什么? 因为 $e_k = x^k - x^*$, 所以:

$$x^k - e_k = x^k - (x^k - x^*) = x^*.$$

也就是说, **Newton 步在最优点附近几乎就是直接往最优点走**。但是它不能完全到达, 因为真实函数不一定是二次函数。Taylor 展开会有余项 r_k , 这个余项满足:

$$\|r_k\| \leq C_1 \|e_k\|^2.$$

所以最后剩下的误差也就是平方级别的：

$$\|e_{k+1}\| \leq C\|e_k\|^2.$$

Newton 法用 *Hessian* 反过来校正梯度，把当前误差里最大、最主要的那部分消掉了。剩下的误差只来自 Taylor 近似不完全准确的余项，而这个余项比原误差小一个量级

说到这里，我们再来回顾一下**梯度下降算法**，在凸函数是 L -光滑且强凸的时候，梯度下降算法可以做到**线性收敛**：

定理 3.3: 强凸光滑下的线性收敛

设 f 为 μ -强凸且 L -光滑，取梯度下降步长 $\alpha = 1/L$ 。则

$$f(x^k) - f^* \leq \left(1 - \frac{\mu}{L}\right)^k (f(x^0) - f^*).$$

很显然牛顿法在最优点附近，**收敛速度显著快于梯度下降法**，但有舍就有的，牛顿法的代价就是每一步都需要昂贵的**二阶信息**（海森矩阵）计算，以及线性代数的计算。

步骤	典型代价	说明
计算梯度 g_k	依问题而定	一阶信息
计算 Hessian G_k	约 $\mathcal{O}(n^2)$	存储也需 $\mathcal{O}(n^2)$
解 $G_k d^k = -g_k$	约 $\mathcal{O}(n^3)$	稠密 Cholesky 或 LU
线搜索	多次函数值计算	提高全局稳定性

求解牛顿方程与精确性：

Cholesky 分解（正定时常用）

Newton 法每一步需要解线性方程组：

$$G_k d^k = -g_k.$$

理论上可以写作 $d^k = -G_k^{-1} g_k$ ，但实际算法中通常不直接求逆，而是直接解线性系统。当 $G_k \succ 0$ ，即 Hessian 是正定矩阵时，可以进行 Cholesky 分解：

$$G_k = LL^T.$$

其中 L 是下三角矩阵，且对角线元素为正。正定这个条件很重要。它不仅保证 Newton 方向是下降方向，也保证 *Cholesky* 分解可以稳定进行。

把 Newton 方程

$$G_k d^k = -g_k$$

改写成：

$$LL^T d^k = -g_k.$$

引入中间变量 $y = L^T d^k$ ，于是分两步求解：

$$Ly = -g_k, \quad L^T d^k = y.$$

第一步解下三角系统，第二步解上三角系统。这样避免了显式计算 G_k^{-1}

精确性：

允许 $G_k d^k + g_k$ 不等于 0，但它必须相对于当前梯度 $\|g_k\|$ 足够小。一个常见做法是：

η_k 逐渐变小。

也就是：

- 远离最优点时，取较大的 η_k ，比如 0.5、0.1，方向大致对即可；
- 接近最优点时，取较小的 η_k ，比如 10^{-2} 、 10^{-4} ，让 Newton 方程解得更准。

这背后的逻辑是：

离**最优点远**时，外层点本身还很**粗糙**，没必要在内层线性方程组上花太多计算；离**最优点近**时，每一步 Newton 方向都**很有价值**，值得**解得更准确**。

怎么理解残差

$G_k d^k + g_k$ 衡量“这个 d^k 离真正 Newton 方程的解还差多少”。 η_k 越小，说明线性系统解得越精确。

共轭梯度法：

它在做什么

CG 不直接分解矩阵，而是不断改进 d ，让残差 $Gd + g$ 逐步变小。

为什么有用

它主要需要矩阵-向量乘法 Gv ，适合大规模稀疏问题，甚至不必显式存下完整 Hessian。

剩下的看PPT

梯度下降法与牛顿法对比：

方法	优点	局限
梯度下降	每步便宜，只需梯度；实现简单；适合大规模问题。	对病态问题敏感，收敛速度依赖条件数。
牛顿法	利用二阶曲率；局部二次收敛；对二次函数一步收敛。	需要 Hessian；每步代价高；远离最优点时需阻尼或修正。

梯度下降法计算代价小，但是速度慢，牛顿法计算代价大，但是在离最优点比较近的时候是二次收敛，找到最优解的速度非常快！

拟牛顿法：

[拟牛顿法.pdf](#)

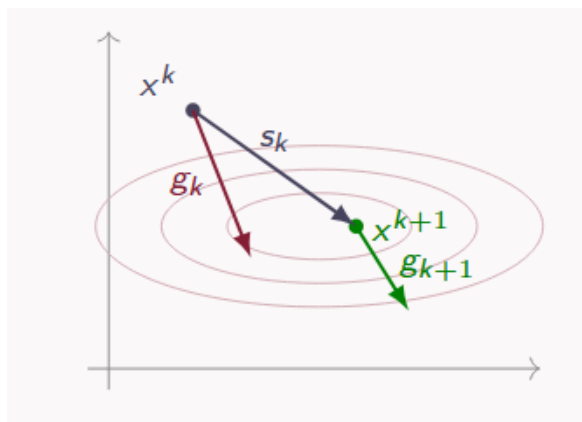
割线条件：

拟牛顿法的目标是：**不直接计算 Hessian 矩阵，却尽量获得类似牛顿法的曲率信息。** 牛顿法使用真实 Hessian $\nabla^2 f(x_k)$ ，方向为 $d_k = -[\nabla^2 f(x_k)]^{-1}g_k$ ，但 Hessian 的计算、存储和求解代价都很高。

拟牛顿法因此改为构造近似矩阵 B_k 或 H_k , 分别近似 *Hessian* 和逆 *Hessian*, 即 $B_k \approx \nabla^2 f(x_k)$, $H_k \approx [\nabla^2 f(x_k)]^{-1}$ 。

从第 k 步走到第 $k+1$ 步, 定义:

$$s_k = x_{k+1} - x_k, \quad y_k = \nabla f(x_{k+1}) - \nabla f(x_k).$$



其中, s_k 表示这一步的位置变化, 也就是“走了多远、往哪个方向走”; y_k 表示这一步的梯度变化, 也就是“走完之后坡度发生了怎样的变化”。

从直觉上来说, 拟牛顿法其实就是在用位置变化和梯度变化来近似估计局部曲率。

为什么这个叫做割线性质? 为什么这个又是合理的? 我们可以先用一维上的性质来理解:

对于一维函数 $f(x)$, 二阶导数可以用导数变化率近似:

$$f''(x) \approx \frac{f'(x_{k+1}) - f'(x_k)}{x_{k+1} - x_k}.$$

这其实就是用导函数 $f'(x)$ 上两个点之间的割线斜率来近似二阶导数。令 $s_k = x_{k+1} - x_k$, $y_k = f'(x_{k+1}) - f'(x_k)$, 那么一维中可以理解为 $f''(x) \approx y_k/s_k$, 等价写成 $B_{k+1}s_k = y_k$ 。

多维时, 对应关系变成:

$$\text{Hessian 近似矩阵} \times \text{位置变化} = \text{梯度变化}$$

定义 2.1: 割线条件

若 B_{k+1} 近似 Hessian，则希望它满足

$$B_{k+1}s_k = y_k.$$

若 H_{k+1} 近似逆 Hessian，则希望它满足

$$H_{k+1}y_k = s_k.$$

如果用 B_{k+1}

B_{k+1} 近似 Hessian。

它把“位置变化” s_k 映射成“梯度变化” y_k 。

如果用 H_{k+1}

H_{k+1} 近似逆 Hessian。

它把“梯度变化” y_k 映射回“位置变化” s_k 。

只有割线条件不够：

根据我们刚刚的方法，我们只能列出一个向量方程，然而这并不能让我们算出一个唯一的解，既然解不唯一，那我们肯定还要设计一套法则，在多个解中调出最好的那个解。

拟牛顿更新的设计目标

- ▶ 满足割线条件，保留最新曲率信息；
- ▶ 尽量少偏离旧矩阵 B_k 或 H_k ；
- ▶ 保持对称性；
- ▶ 在适当条件下保持正定性，保证下降方向。

尽量少改动，指的就是我们希望 B_{k+1} 离 B_k 不要太远，每一步只让我们获得了一小段新的曲率信息，我们当然不希望这个直接把前面的曲率完全推翻，而是希望能在旧矩阵的基础上做一个更新修正。

保持正定性的理由就是希望能够让方向是下降方向，因为拟牛顿法只是在找“海森矩阵”的地方设计了优化方法，其他地方与牛顿法是一样的，而牛顿法在海森矩阵正定时方向是下降方向，这里也是一样的道理。

拟牛顿法的基本框架：

- 计算当前点梯度 g_k

- 用近似矩阵求拟牛顿方向, 令 $d^k = -H_k g_k$
- 线搜索得到步长 α_k
- 更新 $x^{k+1} = x^k + \alpha_k d^k$
- 然后计算 $s_k = x^{k+1} - x^k$, $y_k = g_{k+1} - g_k$
- 更新逆近似矩阵 $H_{k+1} y_k = s_k$

拟牛顿法一般会从初始矩阵 $H_0 = I$ 开始。这样简单、稳定, 而且不需要额外估计初始 Hessian 信息。

此时第一步搜索方向为 $d_0 = -H_0 g_0 = -g_0$, 也就是说, **拟牛顿法的第一步退化成普通负梯度方向**。

直观过程

拟牛顿法一开始像 GD; 随着迭代进行, 它不断积累 (s_i, y_i) , 方向逐渐带有曲率校正。

BFGS 方法逆 Hessian 更新:

BFGS 的核心公式是:

$$H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T,$$

其中: $\rho_k = \frac{1}{y_k^T s_k}$ 。

H_k 是旧的逆 Hessian 近似, H_{k+1} 是更新后的逆 Hessian 近似, s_k, y_k 是当前这一步得到的新曲率信息。

这个更新只使用 H_k, s_k, y_k , 不需要计算真实 Hessian。

- 第一, **保留旧信息**。公式从旧矩阵 H_k 出发, 而不是每一步重新构造一个矩阵。这说明 BFGS 会保留过去迭代中**积累下来的曲率信息**。
- 第二, **写入新曲率**。公式使用当前这一步的 s_k, y_k , 并使更新后的矩阵**满足割线条件**: $H_{k+1} y_k = s_k$, 这意味着, 新的 H_{k+1} 至少在**刚刚观察到的方向上是合理的**。
- 第三, **保持稳定性**。在曲率条件 $y_k^T s_k > 0$ 成立时, BFGS 能够保持 $H_{k+1} \succ 0$ 。BFGS 的重要性质是在 $y_k^T s_k > 0$ 时**保持正定性**, 从而继续给出**下降方向**。

课堂口径

如果 $y_k^T s_k > 0$, 说明刚走过的方向上函数像一个向上的碗; BFGS 就能把这种正曲率写入 H_{k+1} 。

优点

- ▶ 不需要显式 Hessian;
- ▶ 方向质量通常明显好于 GD;
- ▶ 局部可达到超线性收敛;
- ▶ 中等规模问题上非常有效。

局限

- ▶ 仍需存储 $n \times n$ 矩阵;
- ▶ 每步矩阵向量乘法约 $O(n^2)$;
- ▶ 对超大规模问题仍可能太贵。

L-BFGS 法

BFGS 需要完整保存 H_k , 而 L-BFGS 不保存 H_k 。它只保存最近 m 组历史信息:

$$(s_{k-m}, y_{k-m}), \dots, (s_{k-1}, y_{k-1}).$$

其中 $s_i = x_{i+1} - x_i$, $y_i = g_{i+1} - g_i$, 也就是说, L-BFGS 只记得最近几步的“位置变化”和“梯度变化”。 m 通常远小于 n , 例如 $5 \sim 20$, 存储代价因此从 $O(n^2)$ 降为 $O(mn)$ 。

第一轮

从当前梯度 g_k 出发, 按时间倒序“撤销”最近几次曲率影响。

第二轮

再按时间正序把这些曲率信息加回来, 得到近似的 $H_k g_k$ 。

方法	保存内容	适合规模
BFGS	完整矩阵 $H_k \in \mathbb{R}^{n \times n}$	中等规模, 变量数不太大
L-BFGS	最近 m 组向量 (s_i, y_i)	大规模, 变量数很大

方法	使用信息	每步代价	适用场景
GD	梯度	低	大规模、精度要求一般
Newton	梯度 + Hessian	高	中小规模、高精度
BFGS	梯度 + 曲率近似	中高	中等规模光滑优化
L-BFGS	梯度 + 有限记忆曲率	中低	大规模光滑优化